

CommitFlow: Semantic Commitment Verification and Local Correction for Long-Horizon Robot Manipulation VLA Execution

Zixiang Zhao, Yansong Feng, Yang Yang, Chaoyu Wang,
Haoran Xiao, Hui Zhang, Chuang Cheng*, and Jianjun Ma
National University of Defense Technology, Changsha, China

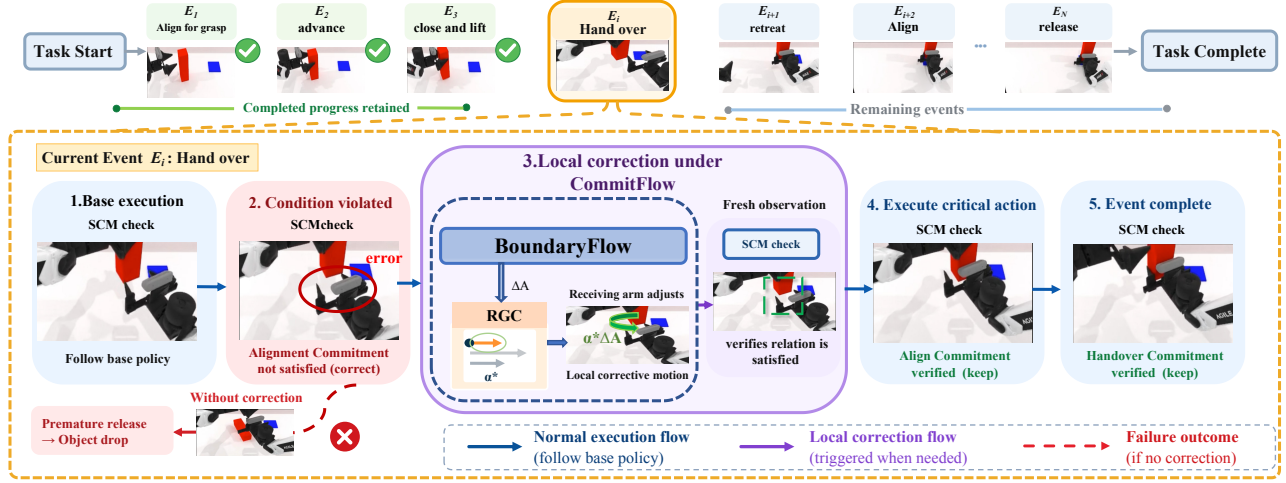


Fig. 1. **Event progression and local correction in CommitFlow.** During handover, the sender retains the object while the receiver adjusts its alignment. BoundaryFlow supplies a local correction, RGC scales it, and SCM re-verifies the physical conditions from fresh observations before dependent actions resume.

Abstract—Although vision-language-action (VLA) policies have advanced rapidly, long-horizon execution may still progress to the next task stage before the required physical effect has been established. We call this a mismatch between semantic commitments—physical conditions that a stage must establish or maintain—and the actual physical state. Because an action command alone cannot confirm such a condition, local deviations can propagate and cause task failure. To address this problem, we present CommitFlow, a closed-loop execution framework that combines commitment monitoring with local correction while keeping the base policy frozen. CommitFlow integrates three components. A Semantic Commitment Monitor (SCM) compares stage requirements against current state evidence and holds back dependent actions when a required condition is unmet or violated. BoundaryFlow then generates a local correction conditioned on the current state and base action, and Relation and Gain Calibration (RGC) selects the smallest correction strength that satisfies the relevant constraints. Across the ten common RoboTwin 2.0 benchmark tasks, CommitFlow achieves a mean success rate of 75.9%, improving on the base policy $\pi_{0.5}$ by 22.7%. Cross-policy experiments show consistent gains, pointing toward reliable long-horizon robot execution.

Keywords: vision-language-action models, semantic commitments, local correction.

*Corresponding author: Chuang Cheng.

I. INTRODUCTION

Vision-Language-Action (VLA) models [1], [2] map visual observations, language instructions, and robot states directly to robot actions, enabling robots to perform a wide range of manipulation tasks. However, extending VLA models to long-horizon tasks remains challenging. Unlike short-horizon tasks, long-horizon tasks typically involve multiple interdependent execution stages, where the outcome of the current stage determines whether subsequent stages can proceed correctly. More specifically, during execution, the current stage may fail to achieve its intended physical effect, while the policy continues to advance through the task sequence and may even transition to the next stage before the current one is properly completed. In such cases, subsequent actions are executed from an incomplete or incorrect physical state, causing local execution deviations to accumulate and propagate across stages, ultimately leading to task failure.

This mismatch can arise when stage transitions occur before physical preconditions are met or when concurrent actions interfere. Figure 1 illustrates the handover case: the sender retains the object while the receiver corrects its alignment, and release waits for verified receiver grasp. Other failures include release outside the placement region or premature basket grasping during ongoing placement. Reliable execution

therefore requires verification of physical preconditions and action dependencies before advancing.

To address this issue, existing studies typically detect anomalies using policy features, visual evidence, or constraint monitoring, and recover execution through replanning or retries. However, these methods often intervene only after clear signs of task failure have emerged. At that point, recovery must deal with a more complex failure state and may also disrupt previously established grasping, support, or interaction relationships. Therefore, we argue that reliable long-horizon execution should not wait until local deviations have evolved into task failures before initiating recovery. Instead, intervention should be moved ahead of critical stage transitions, so that deviations can be identified and corrected before adverse consequences arise and propagate to subsequent stages.

CommitFlow is built around this principle. It represents the physical conditions required for stage progression as semantic commitments and organizes execution as a closed loop of verification, correction, and re-verification. The Semantic Commitment Monitor (SCM) evaluates whether the stage-specific physical conditions are satisfied based on the current observation. When a required condition is not met, SCM postpones downstream actions that depend on it while preserving unaffected execution progress. For the detected local deviation, BoundaryFlow then corrects the action proposed by the base policy without replacing the underlying task policy. The Relation and Gain Calibration (RGC) module further determines the necessary correction magnitude according to the current relational deviation, thereby avoiding excessive intervention. After the correction is executed, SCM re-evaluates the relevant conditions using the updated observation, and task execution proceeds only when the corresponding semantic commitment has been re-established.

Overall, this work addresses early intervention and local correction for long-horizon VLA execution through three aspects:

1. **Semantic commitment monitoring.** We propose SCM to identify unsatisfied physical conditions through state estimation and commitment verification, defer dependent actions, and preserve valid progress.
2. **Learned local correction and gain calibration.** We propose BoundaryFlow to learn local updates from base actions and expert corrections, and RGC to build stage-wise reference ranges from successful data and select correction strength.
3. **Cross-policy evaluation and correction reuse.** We evaluate performance across different base policies and examine the transfer of correction strategies to similar tasks.

II. RELATED WORK

Research on reliable long-horizon VLA execution has mainly progressed along three directions: monitoring task states and physical conditions to assess execution status, handling execution deviations through active recovery and learned action correction, and directly guiding or adjusting

actions at inference time. Together, these studies extend the ability of VLA policies to perceive and intervene during execution from different perspectives.

A. Task-State Perception and Physical-Condition Monitoring

SAFE estimates cross-task failure risk from internal VLA representations [3]. DoReMi formulates plan-execution inconsistencies as constraint violations [4], while ReViP addresses false completion before the intended goal has been achieved [5]. More recently, PhysReflect-VLA evaluates candidate-action feasibility during execution and uses structured self-reflection to guide subsequent action sampling [6]. These studies advance execution monitoring from global risk estimation toward explicit execution-state verification. CommitFlow focuses on stage-dependent physical requirements represented as commitments that gate dependent actions and are re-verified after localized correction.

B. Active Recovery and Learned Action Correction

Once an execution deviation has been detected, restoring task progress constitutes another major line of research.

REFLECT uses multimodal execution history to interpret failures and guide subsequent planning [7], while AIC adjusts manipulation poses online through interactive feedback [8]. FPC-VLA introduces a VLM-based supervisor that evaluates action viability at keyframes and generates language corrections specifying direction and magnitude when potential failures are identified [9]. For longer-horizon tasks, CycleVLA combines progress estimation with termination prediction and uses backtracking near stage transitions [10], while FLARE handles deviations through Retry and Reset behaviors [11]. RePO-VLA further learns corrective behaviors from successful, failed, and recovery trajectories [12]. Subtask-level backtracking or skill resets can also roll back execution components that remain valid, potentially disrupting established progress. FPC-VLA anticipates action failure at keyframes; CommitFlow instead verifies the current stage’s physical commitments and selectively withholds dependent actions until the required conditions are established.

C. Inference-Time Action Guidance and Execution Adjustment

Beyond explicit monitoring and recovery, another line of work directly improves actions generated by the base policy at inference time. V-GPS ranks candidate actions using an offline value function [13], TACO selects more reliable action chunks according to support from successful demonstrations [14], and CAPS corrects drifting trajectories through local sampling and adaptive search [15]. VLA-Corrector detects persistent mismatch between predicted and observed visual evolution, truncates stale actions, and triggers corrective replanning with an adaptive action horizon [16]. Its VLA backbone remains frozen while an external corrector guides the recovery query. These methods improve execution through action selection, local search, or event-triggered replanning. However, trajectory drift and action quality do not by themselves establish whether stage-transition prerequisites are satisfied or which dependent actions should remain withheld until verification.

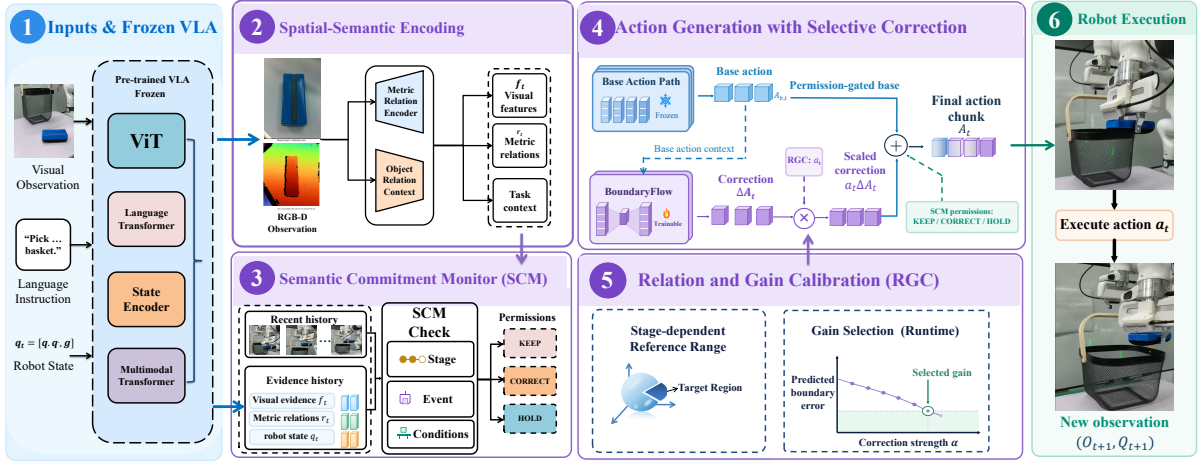


Fig. 2. **CommitFlow overview.** RGB-D encoding supplies visual features, metric relations, and task context. SCM assigns action permissions; BoundaryFlow predicts a base-action-conditioned residual; RGC selects its strength. The permission-gated action is executed, and fresh observations support re-verification.

III. METHOD

Our goal is to enable a VLA policy to identify and correct deviations between intended effects and the actual state during execution, prevent task advancement before the necessary physical conditions are satisfied, and thereby improve system generalization and robustness. To this end, we propose CommitFlow, a closed-loop execution framework built around *semantic commitments* that converts physical-condition deviations detected at runtime into correction targets constrained by current task requirements. SCM verifies the active commitment through lightweight temporal state estimation and passes the missing conditions and existing conditions that must be preserved to BoundaryFlow, enabling it to adjust only the affected portions of the base action. Subsequently, RGC evaluates the predicted states corresponding to different correction strengths using stage reference ranges established from successful execution data and selects a candidate that satisfies both local geometric requirements and action constraints. After the correction is executed, SCM re-verifies the relevant conditions based on new observations and accordingly determines whether to continue correction or resume subsequent actions. For different scenes with the same task structure, the task adapter binds objects and manipulation roles and updates relative states, enabling the commitment-verification and local-correction process to be reused across scene configurations.

Figure 2 summarizes the overall architecture, and Fig. 3 illustrates how BoundaryFlow combines a local correction with the frozen policy’s base action.

A frozen policy receives observations o_t , measured robot state q_t , and instruction ℓ , and proposes

$$A_{B,t} = \pi_B(o_t, q_t, \ell) \in \mathbb{R}^{H \times D_a}. \quad (1)$$

Here H is the horizon and D_a is the action dimension. A *base candidate* is this not-yet-fully-executed chunk. The runtime dispatches only an approved prefix before re-observation.

A. Task-Relevant State and Event Representation

Task-relevant RGB-D features f_t , measured joints, and kinematics provide state evidence. An adapter defines templates for pre-alignment, reaching, grasp closure, and lifting, specifying objects, arm roles, physical conditions, geometric relations, and subsequent actions. After binding perceived objects to the active template, it computes

$$r_t^i = (R_t^w)^\top (p_t^{\text{tgt}} - p_t^{\text{src}}) \in \mathbb{R}^3, \quad (2)$$

where R_t^w is the event-frame orientation and source/target are interaction reference points estimated from RGB-D and kinematics. Orientation and other geometry are represented separately. Visual features condition SCM and BoundaryFlow; metric relations support RGC checks and calibration in physical coordinates. Figure 4(a) reports RGB-D/CAD position errors over all closed-loop frames and the ALIGN TO GRASP subset.

B. Semantic Commitment Monitoring (SCM)

SCM combines a causal state estimator with a commitment verifier. Stage and event predictions establish context; only verified physical conditions authorize progression.

Causal state estimation. A gated recurrent unit (GRU) encodes an L -frame history:

$$x_t = [f_t; r_t^i; q_t], \quad h_t = \text{GRU}(x_{t-L+1:t}). \quad (3)$$

Stage and event softmax heads read out h_t , while a sigmoid head predicts holding, support, contact, and release probabilities. Event predictions are matched to the adapter’s active roles and legal task structure.

History-window supervision. Annotated causal windows train the observer with $\mathcal{L}_{\text{obs}} = \mathcal{L}_{\text{stage}} + \beta_e \mathcal{L}_{\text{event}} + \beta_p \mathcal{L}_{\text{condition}}$: stage/event cross-entropy and physical-condition binary cross-entropy (BCE). Rule-generated labels supply supervision only. Invalid annotations are masked; condition BCE averages valid labels and is zero when none are available. Observer training is separate from (8).

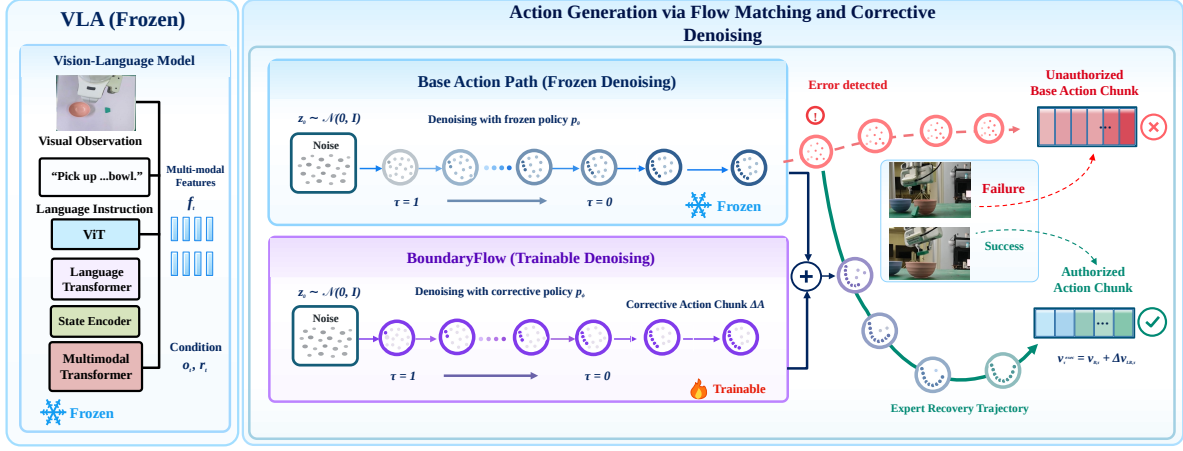


Fig. 3. **BoundaryFlow action generation and local correction.** The frozen VLA generates a base action chunk through flow matching, while the trainable BoundaryFlow branch generates a corrective action chunk. Combining the two produces the corrected action chunk.

Commitments and permissions. For event e_t , SCM checks the adapter’s required conditions C_{e_t} against state estimates, metric relations, and history. Conditions are *pending* until satisfied, *blocking* when required by a dependent action but unsupported by evidence, and *violated* after losing satisfaction. Stage predictions cannot override unresolved commitments.

Before protected actions, unsupported critical actions and their dependent time–channel elements form $\mathcal{H}_t$; the locally adjustable prefix forms C_t :

$$\Gamma_{t,k,d} = \begin{cases} \text{HOLD}, & (k,d) \in \mathcal{H}_t, \\ \text{CORRECT}, & (k,d) \in C_t \setminus \mathcal{H}_t, \\ \text{KEEP}, & \text{otherwise.} \end{cases} \quad (4)$$

HOLD takes priority over local correction, while KEEP retains unaffected base motion.

SCM monitoring. Across 1,971 closed-loop frames from six streams, SCM achieves Macro-F1 scores of **88.98%** for stage recognition and **94.89%** for event-class recognition (Fig. 4(b)).

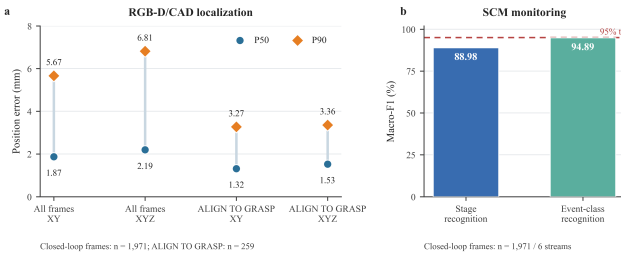


Fig. 4. **RGB-D/CAD localization and SCM recognition performance.** (a) Median (P50) and 90th-percentile (P90) position errors in XY and XYZ over all 1,971 closed-loop frames and the 259-frame ALIGN TO GRASP subset. (b) Macro-F1 for stage and event-class recognition across six streams (1,971 frames). The dashed line marks the 95% target.

C. BoundaryFlow: Base-Action-Conditioned Correction

BoundaryFlow predicts bounded local residuals conditioned on the current state and frozen base proposal.

Conditioning and low-rank correction. The condition z_t combines local visual features, normalized object–arm relations and their protected-boundary predictions, measured joints, the base prefix, and stage/role evidence. Conditioning on the proposed prefix accounts for its effect on the deviation. A fixed active-arm mapping provides shared six-joint coordinates (superscript c) over $K = 10$ steps for compatible arm roles.

An uncentered singular value decomposition of vectorized training expert–base residuals provides the leading $r = 12$ left singular vectors $B_r \in \mathbb{R}^{60 \times r}$. The residual is

$$c_t = c_{\max} \tanh f_{\theta}(z_t), \quad \Delta A_t^c = \text{reshape}(B_r c_t, K, 6). \quad (5)$$

The orthonormal basis restricts residual directions to the training subspace and yields $\|\text{vec}(\Delta A_t^c)\|_2 \leq \sqrt{r} c_{\max}$ in normalized action coordinates.

Boundary-aligned residual supervision. For each recoverable pre-transition perturbation, the frozen base and expert generate paired prefixes from the same state, $A_{B,t}^c$ and $A_t^{\star,c}$. The expert repairs the relation while preserving invariants I_i and withholding the protected effect. With $M^{\text{corr}} = \mathbb{I}[\Gamma = \text{CORRECT}]$, the coefficient target is

$$c_t^{\star} = B_r^{\top} \text{vec}\left(M_t^{\text{corr},c} \odot [A_t^{\star,c} - A_{B,t}^c]\right). \quad (6)$$

Valid nominal states at the same stage anchors receive zero-correction supervision.

Joint action and geometric training. We supervise coefficients against $c_t^{\star}$ with $\mathcal{L}_{\text{coef}}$, corrected actions against the expert on permitted coordinates with $\mathcal{L}_{\text{act}}$, and denoising at the flow interface with $\mathcal{L}_{\text{denoise}}$. For geometric supervision, FK in physical joint coordinates yields $\hat{r}_{t,k}^{\text{corr}}$. Using the RGC margin from (12), let $m_{t,k}^{\text{corr}} = m_i(\hat{r}_{t,k}^{\text{corr}})$, $m_{t,0} = m_i(r_t^i)$, and

$\bar{m}_{t,k} = (1 - k/K)m_{t,0} + (k/K)\delta$. We penalize terminal error and progress below this soft reference:

$$\mathcal{L}_{\text{contract}} = [\delta - m_{t,K}^{\text{corr}}]_+ + \frac{\lambda_p}{K} \sum_{k=1}^K [\bar{m}_{t,k} - m_{t,k}^{\text{corr}}]_+. \quad (7)$$

Each term becomes zero when its margin requirement is met.

Nominal pairs use $\mathcal{L}_{\text{null}} = \|c_t\|_2^2$. The full objective is

$$\mathcal{L}_{\text{BoundaryFlow}} = \mathcal{L}_{\text{coef}} + \lambda_a \mathcal{L}_{\text{act}} + \lambda_d \mathcal{L}_{\text{denoise}} + \lambda_g \mathcal{L}_{\text{contract}} + \lambda_0 \mathcal{L}_{\text{null}}. \quad (8)$$

Repair and null supervision use their respective sample subsets.

Integration with base action generation. For $M^{\text{auth}} = \mathbb{I}[\Gamma \neq \text{HOLD}]$ and candidate gain α , SCM permissions constrain composition:

$$A_t^{\text{gate}} = M^{\text{auth}} \odot A_{B,t} + (1 - M^{\text{auth}}) \odot A_t^{\text{hold}}, \quad (9)$$

$$A_t^{(\alpha)} = A_t^{\text{gate}} + \alpha M^{\text{corr}} \odot I_a(\Delta A_t^c).$$

The map I_a restores full action coordinates. A_t^{hold} retains the required gripper state and measured joint positions (or zero increments/velocities); RGC selects α via (13).

For flow policies, apply (9) to the clean-action estimate $\bar{A}_B^\tau = x_\tau - \tau v_B^\tau$, with noisy action x_τ , then map $\bar{A}_{\text{exec}}^\tau$ back to the sampling field:

$$v_{\text{exec}}^\tau = \frac{x_\tau - \bar{A}_{\text{exec}}^\tau}{\tau} = v_B^\tau - \frac{\bar{A}_{\text{exec}}^\tau - \bar{A}_B^\tau}{\tau}, \quad \tau > 0. \quad (10)$$

For $\tau > 0$, the mapping recovers v_B^τ whenever $\bar{A}_{\text{exec}}^\tau = \bar{A}_B^\tau$.

SCM re-verifies stage conditions and retained states from fresh measurements before discarding stale actions and re-querying the base policy (Algorithm 1).

D. Relation and Gain Calibration (RGC)

RGC calibrates reference regions offline and selects gain along BoundaryFlow’s predicted residual.

Successful-state reference regions. Successful boundary states are grouped by approach direction and action configuration, retaining grasp-specific labels and normal pre-approach axial stand-off. A fitting split provides coordinatewise medians μ_i and per-axis standard deviations S_i with positive floors; an independent success-calibration split gives

$$d_n = \|S_i^{-1}(r_n^i - \mu_i)\|_\infty, \quad q_i = \text{Quantile}_{0.95}(\{d_n\}). \quad (11)$$

Using ordered sample $[0.95n_i]$ and $D_i = q_i S_i$ (positive floor on q_i), define

$$m_i(r) = 1 - \|D_i^{-1}(r - \mu_i)\|_\infty, \quad (12)$$

$$\mathcal{R}_i = \{r : m_i(r) \geq 0\}, \quad \mathcal{R}_i^\delta = \{r : m_i(r) \geq \delta\}.$$

Fitting and evaluation use disjoint scenes. P95 calibrates the reference region’s empirical coverage. SCM verifies holding and support from observed evidence. At the checkpoint, valid base execution uses $\mathcal{R}_i$, and repair targets $\mathcal{R}_i^\delta$ with $0 < \delta < 1$.

Candidate geometry and checkpoints. Gripper changes, low-speed locations, and kinematic directions identify checkpoints. FK of the unexecuted prefix gives a *boundary residual*

Algorithm 1 Commitment Monitoring and Local Correction

One call per monitoring cycle; history and unresolved commitments persist across calls.

Input: $A_{B,t}$, observation history, E_i , (μ_i, D_i) , $\mathcal{A}$
1: Perceive and estimate semantic state (Secs. III-A–III-B).
2: Locate checkpoint; compute BRT (Sec. III-D).
3: Verify commitments; assign Γ_t via (4).
4: **if** $\exists(k, d) : \Gamma_{t,k,d} = \text{CORRECT}$ **then**
5: $(c_t, \Delta A_t^c) \leftarrow \text{BoundaryFlow}(z_t)$, (5).
6: Form candidates and $\mathcal{A}_t^{\text{ok}}$ via (9) and (13).
7: **if** $\mathcal{A}_t^{\text{ok}} \neq \emptyset$ **then**
8: $\alpha_t \leftarrow \min \mathcal{A}_t^{\text{ok}}$; ExecutePrefix($A_t^{(\alpha_t)}$).
9: **else** preserve valid motion; retain affected holds.
10: **end if**
11: **else** ExecutePrefix(A_t^{gate}) from (9).
12: **end if**
13: Refresh observations/history; re-verify before resuming held actions.
14: Advance only on event completion and successor readiness.

trace (BRT) for short-horizon geometric screening. A base prefix may continue if it can establish the relation before the checkpoint. Fresh observations verify the required condition before crossing.

Runtime gain selection. For one predicted residual, RGC checks permission-respecting candidates from (9) over an ordered set $\mathcal{A} \subset (0, 1]$:

$$\mathcal{A}_t^{\text{ok}} = \left\{ \alpha \in \mathcal{A} : m_i(\hat{r}_{\text{end}}^{(\alpha)}) \geq \delta, \mathcal{K}_i(A_t^{(\alpha)}) = 1 \right\}, \quad (13)$$

Here $\mathcal{K}_i$ checks joint, step-size, and permission constraints. When a feasible gain exists, RGC selects $\min \mathcal{A}_t^{\text{ok}}$, and the runtime executes its prefix. If the feasible set is empty, SCM retains affected holds and refreshes evidence or regenerates the correction. Gains and references are fixed before evaluation; checks repeat after new observations. Valid base execution continues under SCM permissions.

IV. EXPERIMENTS

We evaluate the effects of commitment monitoring and local correction on task-level performance, their applicability across base policies, and their benefits over re-querying or inverse kinematics, and examine correction reuse across related interactions.

Table I includes the shared benchmark results reported in CAPS [15].

A. Experimental Setup

We evaluate 11 long-horizon interactive tasks from Robo-Twin 2.0 [17], covering handover, hanging, directional placement, and multi-target placement. Each task is evaluated over 100 trials with different random seeds. Table I reports the mean success rate across 10 common tasks. The cross-policy evaluation in Table II averages all 11 tasks. The main comparisons include RDT [18], $\pi_{0.5}$ [19], TACO, CAPS, and CommitFlow. We also integrate CommitFlow with π_0 [20] and LingBot-VLA [21] to assess its applicability across base policies. During CommitFlow training and evaluation, all base-policy parameters remain frozen, and the corresponding BoundaryFlow correction head is configured according to the base policy’s action representation.

TABLE I
ROBOTWIN 2.0 RESULTS (%)

Method	Handover Block	Handover Mic	Hang Mug	Bread Basket	Bread Skillet	Place A2B Left	Place A2B Right	Object Basket	Object Stand	Object Scale	Object Cabinet	Avg. ₁₀
RDT	45	90	1	10	5	3	1	33	15	1	33	23.6
$\pi_{0.5}$	24	58	11	62	62	51	39	69	68	45	54	53.2
$\pi_{0.5}$ +TACO [14]	36	63	—	65	61	56	42	78	78	52	56	58.7
$\pi_{0.5}$ +CAPS [15]	38	72	—	63	69	54	47	76	81	57	58	61.5
$\pi_{0.5}$ +CommitFlow	66	60	42	75	78	70	69	88	97	85	71	75.9

Task success rates over 100 trials per task with randomized scenes. Avg.₁₀ averages the 10 common tasks, excluding Hang Mug. Bold marks column best; “—” denotes an unreported value.

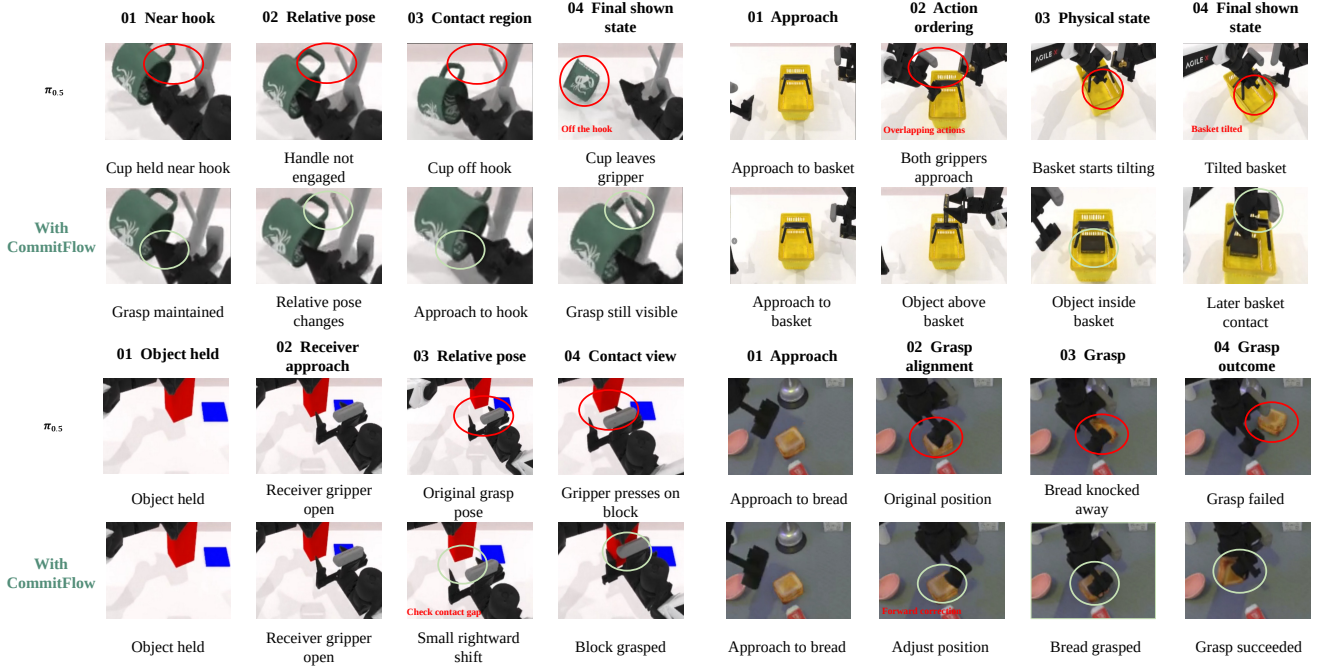


Fig. 5. **Qualitative execution sequences in simulation.** Four examples compare the base policy (upper rows) with CommitFlow (lower rows). Frames progress from left to right; marked regions highlight the relevant physical relations.

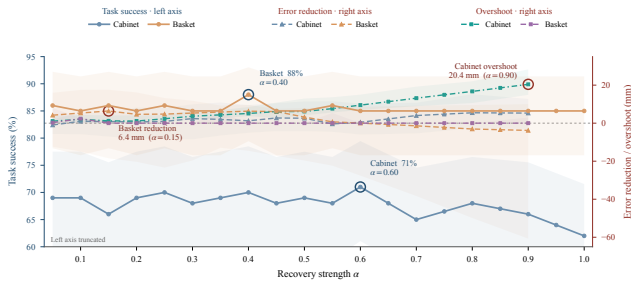


Fig. 6. **Recovery-strength sweep.** Cabinet and Basket: overall task success (left axis), recovery-error reduction and zero-crossing overshoot in mm (right axis). Circled annotations highlight selected measurements; the success axis is truncated.

All policies share the relation-verification and action-permission logic; compatible correction heads adapt the output to each policy’s action representation. BoundaryFlow uses a 10-step, six-joint window and a rank-12 basis.

TABLE II
CROSS-POLICY RESULTS (%)

Task	π_0 [20]	$\pi_{0.5}$ [19]	LingBot-VLA [21]
Handover Block	26 → 45	24 → 66	85 → 90
Handover Mic	86 → 89	58 → 60	95 → 96
Hang Mug	3 → 25	11 → 42	49 → 62
Bread Basket	35 → 50	62 → 75	89 → 91
Bread Skillet	12 → 23	62 → 78	80 → 83
Place A2B Left	30 → 45	51 → 70	83 → 83
Place A2B Right	28 → 38	39 → 69	82 → 84
Object Basket	50 → 73	69 → 88	73 → 94
Object Stand	60 → 80	68 → 97	95 → 96
Object Scale	53 → 73	45 → 85	96 → 93
Object Cabinet	18 → 41	54 → 71	68 → 81
Avg. ₁₁	36.5 → 52.9	49.4 → 72.8	81.4 → 86.6

Success rates: Base → Base+CommitFlow. Bold marks improvement. Avg.₁₁ averages all 11 tasks. Each policy uses a compatible correction head.

B. Task Performance

Table I reports a ten-task mean success rate of 75.9% for CommitFlow, compared with 53.2%, 58.7%, and 61.5% for $\pi_{0.5}$, TACO, and CAPS. Improvements span directional place-

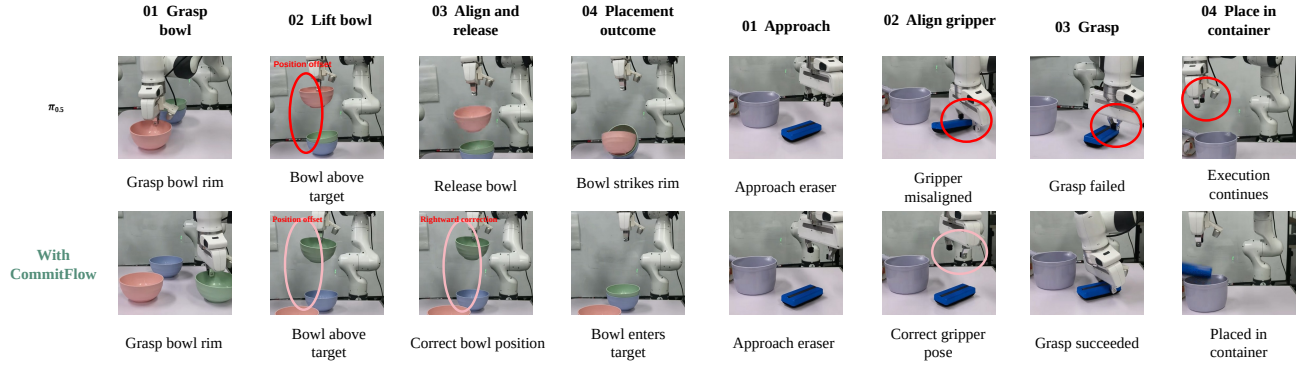


Fig. 7. **Qualitative execution sequences on the real robot.** Bowl stacking (left) and eraser placement (right), comparing the base policy (upper row) with CommitFlow (lower row). Frames progress from left to right.

TABLE III
LOCAL CORRECTION ABLATION (%)

Task	Fresh Base	Fresh + IK	Fresh + Ours
Object Cabinet	61	57	71
Object Basket	60	68	88
Place A2B Left	55	57	70
Place A2B Right	56	60	69
Object Scale	78	78	85
Object Stand	91	94	97
Avg.	66.8	69.0	80.0

Task success rates. All variants re-query Base; Avg. is the six-task macro-average.

ment, object placement, and handover; for example, Handover Block increases from 24% to 66%. These results support verifying physical relations and applying local correction before execution errors propagate to later stages.

Table II reports results across all 11 tasks: Hang Mug rises from 3% to 25% with π_0 , while Object Basket and Object Cabinet increase from 73% to 94% and 68% to 81% with LingBot-VLA. Gains are not universal: LingBot-VLA remains at 83% on Place A2B Left and decreases from 96% to 93% on Object Scale. The shared execution mechanism thus improves performance without guaranteeing gains on every policy-task pair.

Figure 5 compares the base policy and CommitFlow in four simulated examples: mug-hook alignment, placement-grasp ordering, block handover, and bread grasping. Successive observations are arranged from left to right.

C. Ablation Experiments

We ablate the local-correction method and correction strength. With all variants re-querying the base policy, Fresh + Ours achieves 80.0% mean success across six tasks, versus 66.8% for Fresh Base and 69.0% for Fresh + IK (Table III). IK reduces Object Cabinet success from 61% to 57%, suggesting that geometric alignment alone may not restore compatibility with subsequent base-policy actions.

Figure 6 reports 71% success on Cabinet at $\alpha = 0.60$ and 88% on Basket at $\alpha = 0.40$. Larger correction strengths increase Cabinet overshoot, reaching 20.4 mm at $\alpha = 0.90$. This motivates balancing relation recovery against disruption

of effective base motion. RGC scales BoundaryFlow’s predicted correction direction according to the current state, so its corrective capacity remains bounded by the local residual representation. Such corrections may be insufficient for complex contact dynamics, obstacle avoidance, regrasping, or global trajectory planning.

Among recovery-triggered samples involving regrasping after alignment correction, local recovery and final task success rates are 45.8% (109/238) and 18.1% (43/238) for π_0 , and 66.7% (100/150) and 46.7% (70/150) for $\pi_{0.5}$. Thus, successful local grasp recovery does not guarantee success in subsequent task stages. CommitFlow repairs local deviations while preserving valid progress, but cannot provide task skills absent from the base policy.

D. Local-Correction Reuse Across Objects

We evaluate whether learned local corrections can be reused across tasks with the same interaction structure but different objects. For handover, a BoundaryFlow head trained only on Handover Block correction samples is applied to Handover Mic without target-task correction demonstrations. The correction head remains unchanged, while the object-receiver relation reference is rebound to the new interaction. Similarly, for Can $\rightarrow$ Basket, we reuse the correction head trained on Object Basket and bind it to the corresponding can-basket relation.

Table IV summarizes both correction-reuse evaluations in simulation. Success increases from 58% to 65% on Handover Mic and from 40% to 58% on Can $\rightarrow$ Basket. These results indicate that relation-conditioned local corrections can remain effective after object substitution, supporting reuse of an existing correction head across related interactions rather than requiring a separately trained correction module for every object configuration.

E. Real-World Experiments

We further evaluate CommitFlow on two real-world manipulation tasks, with 100 trials per task. As shown in Table IV, CommitFlow improves the success rate of $\pi_{0.5}$ from 50% to 63% on Place Blackboard Eraser in Container (Eraser $\rightarrow$ Container) and from 40% to 50% on Stack Three Bowls.

TABLE IV
CORRECTION REUSE AND REAL-WORLD RESULTS (%)

Task	Base	+ CommitFlow
Correction reuse — simulation		
Handover Mic	58	65
Can → Basket	40	58
Real-world execution		
Eraser → Container	50	63
Stack Three Bowls	40	50

Task success rates. Base denotes $\pi_{0.5}$.

Figure 7 further illustrates how these improvements arise during execution. In bowl stacking, the base policy releases while the bowl remains offset from the target, causing contact with the rim, whereas CommitFlow corrects the bowl position before release. In eraser placement, gripper misalignment causes the base policy to miss the grasp, while CommitFlow adjusts the gripper pose before closure and subsequently completes the placement. These examples illustrate pre-transition intervention: local geometric deviations are corrected before critical dependent actions such as grasp closure or release are allowed to proceed.

V. CONCLUSION

We presented CommitFlow, which verifies semantic commitments and combines local correction with gain calibration before critical stage transitions. Across simulation and real-world tasks, the framework improves execution with frozen base policies, while correction-reuse experiments support transfer across related interactions. Its central principle is to repair unmet physical conditions while preserving valid progress and to re-verify those conditions using fresh observations before dependent actions resume.

Runtime verification remains dependent on RGB-D perception and temporal state estimation. Occlusion, invalid depth, object-state estimation errors, and temporal asynchrony can compromise the estimated relations and physical conditions. Task-dependent event and precondition definitions also limit extension to open-ended tasks and unfamiliar interaction relations. Future work will explore adaptive task adapters that infer event structures, manipulation roles, and physical preconditions from observations and language instructions. Execution feedback will guide adapter refinement and condition verification under perceptual uncertainty. We will also investigate online calibration of stage reference regions and verification thresholds using verified execution outcomes. Conservative updates will aim to accommodate changing objects and sensing conditions while preserving the closed loop of verification, correction, and re-verification.

REFERENCES

- [1] B. Zitkovich, T. Yu, S. Xu, et al., “RT-2: Vision-language-action models transfer web knowledge to robotic control,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 2165–2183.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, et al., “OpenVLA: An open-source vision-language-action model,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 2679–2713.
- [3] Q. Gu, Y. Ju, S. Sun, I. Gilitschenski, H. Nishimura, M. Itkina, and F. Shkurti, “SAFE: Multitask failure detection for vision-language-action models,” in *Advances in Neural Information Processing Systems*, vol. 38, 2025.
- [4] Y. Guo, Y.-J. Wang, L. Zha, and J. Chen, “DoReMi: Grounding language model by detecting and recovering from plan-execution misalignment,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2024.
- [5] Z. Li, Y. Li, J.-J. Jiang, L. Zhou, T. Zhang, J. Yin, M. Lin, Y.-L. Wei, and W.-S. Zheng, “ReViP: Mitigating false completion in vision-language-action models with vision-proprioception rebalance,” *arXiv preprint arXiv:2601.16667*, 2026.
- [6] J. Yang, T. Yang, W. Li, X. Chang, F. Chao, C. Shang, and Q. Shen, “PhysReflect-VLA: Physical feasibility and self-reflective regulation for reliable vision-language-action policies,” *arXiv preprint arXiv:2606.27146*, 2026.
- [7] Z. Liu, A. Bahety, and S. Song, “REFLECT: Summarizing robot experiences for failure explanation and correction,” in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229, 2023, pp. 3468–3484.
- [8] C. Xiong, C. Shen, X. Li, X. Zhou, J. Liu, R. Wang, and H. Dong, “Autonomous interactive correction MLLM for robust robotic manipulation,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 3139–3156.
- [9] Y. Yang, Z. Duan, T. Xie, F. Cao, P. Shen, P. Song, C. Zhao, P. Jin, G. Sun, S. Xu, Y. You, and J. Liu, “FPC-VLA: A vision-language-action framework with a supervisor for failure prediction and correction,” *Expert Systems with Applications*, vol. 316, Art. no. 131742, 2026, doi: 10.1016/j.eswa.2026.131742.
- [10] C. Ma, K. Lu, G. Yang, et al., “CycleVLA: Proactive self-correcting vision-language-action models via subtask backtracking and minimum Bayes risk decoding,” *arXiv preprint arXiv:2601.02295*, 2026.
- [11] G. Zhao, Z. Tang, X. Chen, Z. Kuang, Y. Tian, and G. Li, “FLARE: A failure-aware framework for autonomous correction and recovery in visual-language robotic manipulation,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 22 391–22 401.
- [12] W. Liufu, X. Guo, R. Chen, J. Liu, K. Zhang, X. Liang, J. Lin, D. Sun, Y. Wang, R. Xu, B. Lin, B. Yang, T. Cao, B. Peng, D. Zhang, G. Wang, M. Wang, L. Lin, and X. Liang, “RePO-VLA: Recovery-driven policy optimization for vision-language-action models,” *arXiv preprint arXiv:2605.09410*, 2026.
- [13] M. Nakamoto, O. Mees, A. Kumar, and S. Levine, “Steering your generalists: Improving robotic foundation models via value guidance,” in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270, 2025, pp. 4996–5013.
- [14] S. Yang, Y. Zhang, H. He, L. Pan, X. Li, C. Bai, and X. Li, “Steering vision-language-action models as anti-exploration: A test-time scaling approach,” *arXiv preprint arXiv:2512.02834*, 2025.
- [15] K. Chen, Y. Long, and M. Shang, “Drift is a sampling error: SNR-aware power distributions for long-horizon robotic planning,” in *International Conference on Machine Learning*, 2026.
- [16] Y. Pan, M. Pan, Q. Lu, J. Huang, M. Zhang, S. Huang, X. Li, J. Zhang, Y. Shen, X. Zhang, and W. Zhang, “VLA-Corrector: Lightweight detect-and-correct inference for adaptive action horizon,” *arXiv preprint arXiv:2607.01804*, 2026.
- [17] T. Chen, Z. Chen, B. Chen et al., “RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation,” in *International Conference on Machine Learning*, 2026.
- [18] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, “RDT-1B: A diffusion foundation model for bimanual manipulation,” in *International Conference on Learning Representations*, 2025.
- [19] K. Black, N. Brown, J. Darpanian et al., “ $\pi_{0.5}$: A vision-language-action model with open-world generalization,” in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305, 2025, pp. 17–40.
- [20] K. Black, N. Brown, D. Driess et al., “ π_0 : A vision-language-action flow model for general robot control,” in *Robotics: Science and Systems*, 2025.
- [21] W. Wu, F. Lu, Y. Wang et al., “A pragmatic VLA foundation model,” *arXiv preprint arXiv:2601.18692*, 2026.